深度學習 Lab6 實驗報告

基於 ModernBERT 與 LoRA 之序列生成模型

李朋逸 (314831024)

December 7, 2025

一、實驗概述

本次實驗旨在解決文字摘要問題。面對資料風格迥異的挑戰（Reddit 的長篇故事 vs. SAMSum 的破碎對話）。

實驗過程分為三個主要階段，分別對應初期嘗試、問題除錯（Debug）與最終優化。從初期的欠擬合，到中期的 PPL 數值陷阱，最終透過架構調整與推論策略優化，成功將 ROUGE-L 分數從 0.12 提升至 0.187。

二、實驗過程與分析

第一階段：初期嘗試

1. 實驗設定

- **Encoder 策略**：完全凍結 (Frozen)，僅訓練 Decoder 與 LoRA。
- **Data Handling**：直接混合 TIFU 與 SAMSum，未進行權重調整。
- **結果**：Training 起步 PPL 高達 1017，最終卡在 90.38 左右。Kaggle 成績僅 **ROUGE-L: 0.12049**。

2. 失敗分析

ModernBERT 雖然預訓練强大，但對於「摘要」這種特定任務並無先驗知識。完全凍結 Encoder 導致模型無法適應 TIFU 的鄉民梗與 SAMSum 的口語對話，造成嚴重的 Underfitting。

第二階段：架構升級

為了突破瓶頸，我決定對架構進行大幅度升級，並引入更多訓練技巧。

1. 架構調整

1. **Decoder 升級**：測試了 4 到 16 層，最終發現 **8 層**效果最佳；維度擴展至 768×4。
2. **組件更換**：激活函數從 ReLU 改為 **GELU**；Normalization 位置改為 **Pre-Norm** 提升深層網路穩定性。

2. 異常現象

為了防止過擬合，我加入了 **Label Smoothing (0.1)**。結果 Validation PPL 異常飆升（>150）。檢查之後發現，這是因為 Label Smoothing 在計算 Loss 時加入了懲罰項，導致 $PPL = \exp(Loss)$ 數值虛高，無法真實反映模型能力。移除後 PPL 回落至 40 上下，ROUGE-L 提升至 **0.144**。

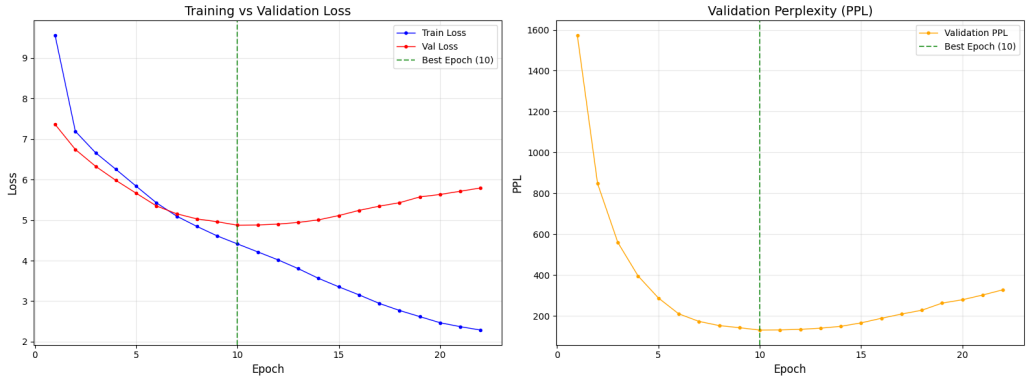

Figure 1: 階段二：Loss 下降曲線與 PPL 變化 (Best Epoch 10)

第三階段：最終優化

1. 學習率策略的反思

在優化過程中，我發現了與傳統理論相悖的現象：

- **Scheduler 的陷阱**：初期程式碼有誤導致 LR 未隨 Epoch 下降。修正後啓用標準 Decay，Validation Loss 反而變差。推測是因為 Dataset 太小，模型需要較高的 LR 能量才能跳出局部最佳解。

- **Encoder/Decoder 共用 LR**：理論上 Encoder 應使用極小 LR。但實驗顯示，**讓 Encoder 與 Decoder 使用相同且較大的 LR (2e-4)** 效果更好。這可能是因為資料集存在巨大的 **Domain Shift**。

2. 訓練策略

- **Unfreezing**：將 Encoder 解鎖層數大幅增加至 **10 層**。
- **Smaller Batch Size**：將 Batch Size 調降至 **16**，意外發現收斂速度變快且效果更好。
- **Loss Weighting**：為了平衡資料量，我引入了動態權重機制（詳見下節）。

3. 損失函數實作細節 (Implementation of Loss Function)

為了滿足助教對於 Loss Function 的要求，並解決資料不平衡問題，本實驗實作了 **Weighted Cross-Entropy Loss** 搭配 **Label Smoothing**。

數學定義：損失函數 L 定義為加權後的 Cross Entropy：

$$L = \frac{\sum_{i=1}^{N} w_i \cdot \ell(x_i, y_i)}{\sum_{i=1}^{N} w_i} \tag{1}$$

其中：

- $\ell(x, y)$ 為包含 Label Smoothing ($\epsilon = 0.1$) 的 Cross Entropy Loss。
- w_i 為樣本權重。為了強迫模型關注長文本，我們將 TIFU 的權重設為 2.0，SAMSum 設為 1.0。

程式碼實作：

```
# 1. 計算基礎 Cross Entropy (含 Label Smoothing)
raw_loss_per_token = F.cross_entropy(
    logits, trg_label, label_smoothing=0.1, reduction='none'
)

# 2. 設定權重 (TIFU x2.0, SAMSum x1.0)
sample_weights = torch.where(is_tifu == 1, 2.0, 1.0)

# 3. 將權重擴展至 Token Level 並計算平均 Loss
token_weights = torch.repeat_interleave(sample_weights, trg_input_lens)
loss = (raw_loss_per_token * token_weights).sum() / token_weights.sum()
```

結果：經過上述調整，Validation PPL 降至 **37.16**，ROUGE-L 提升至 **0.187**。

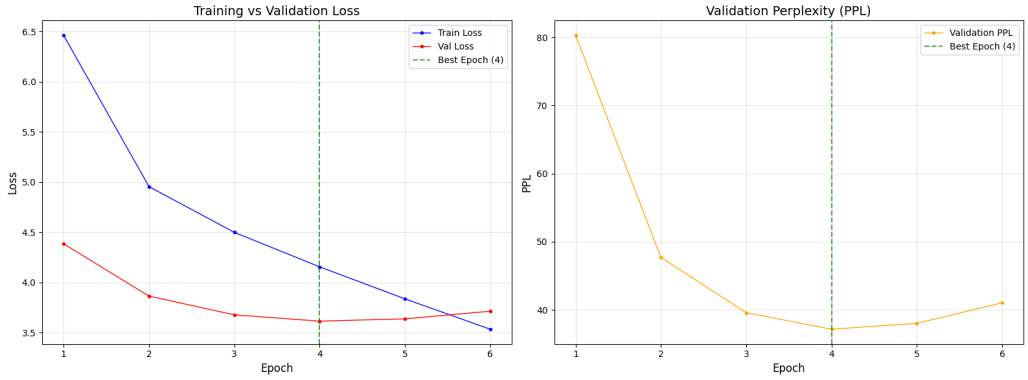

Figure 2: 階段三：最終收斂曲線 (Best Epoch 4, PPL 37.16)

4. 推論階段的關鍵修正

- **關閉 Sampling**：捨棄 `top_k/top_p`，避免隨機性導致生成品質不穩。
- **Beam Search**：針對 TIFU 容易「鬼打牆」的特性，改用 **Beam Size = 3** 搭配 **Repetition Penalty = 1.2**，顯著提升了語句通順度。
- **資料清洗**：修復了 CSV 中的換行符號 (`\n`) 導致的格式錯誤。

四、加分項目與優化

為了進一步提升模型效率與生成速度，本實驗實作了以下進階技術：

1. 訓練流程加速

本實驗採用了多層次的加速技術，確保在有限資源下達到最高效率：

- **Flash Attention 2**:在 Decoder 與 Cross-Attention 層全面引入 `flash_attn_varlen_func` 機制，將注意力機制的複雜度從 $O(N^2)$ 優化，顯著提升長序列（TIFU）的訓練速度並節省顯存。
- **Mixed Precision (BF16)**：全流程採用 `bfloat16` 進行訓練 (`torch.autocast`)，在保持數值穩定性的同時，大幅降低顯存占用並提升矩陣運算速度。
- **LoRA (Low-Rank Adaptation)**：透過凍結大部分參數，僅微調低秩矩陣 ($r = 64$)，大幅減少了梯度計算的開銷。

2. Auto-regression 推論加速

針對 Seq2Seq 生成過程進行了優化，解決了重複運算的效能瓶頸:

- **Encoder Output Caching** :

  在 Inference 階段，針對 Auto-regression 的特性重寫了 `generate` 函數。我們將 Encoder 的運算**移出生成迴圈**，確保對輸入文章（Source Text）只進行一次編碼，並在後續的每一個解碼步驟中重複利用該特徵張量 (`cached_enc_output`)。這避免了每生成一個 Token 就重算一次 Encoder 的巨大開銷，大幅提升了推論速度。

五、結論

本次實驗證明了在混合長短文本的摘要任務中，單純依賴預訓練模型是不夠的。

雖然 SAMSum 部分在獨立測試中可達到極低的 PPL (約 16)，但在混合訓練中，受限於 TIFU 的文本複雜度，整體 PPL 收斂在 37 左右是合理的平衡點。透過 **架構升級 (GELU/Pre-norm)**、**Loss Weighting (Weighted Cross-Entropy)** 以及 **Inference 參數優化**，最終在 Kaggle 上獲得了 **ROUGE-L: 0.187** 的成績。